Process-based evaluation of multi-agent systems

Carolyn McDonald Mill
The Alan Turning Institute
c.mill2@newcastle.ac.uk

Abstract

In this work we examined the ability of multiple large-language models to play the cooperative card game *Hanabi* in multi-agent systems. We adapted the game to allow for a) player-dependent weighted decks that provides each player with a personal game score and b) increased levels of context players can provide when giving hints. We tested 3 different weighting scenarios and 3 different context levels for a total of 9 different game types, to determine how the difference in motivations between more cooperative and competitive games and the difference in the information content in hints would interact. We tested two different models; *Kimi-K2.5* and *phi-4-reasoning*. We found the differences in average score between game types were negligible, potentially due to small sample size. However we an interesting emergent behaviour where *phi-4-reasoning* agents playing competitively-weighted games game less hints to other players on a player's highly weighted deck, implying agents were trying to avoid giving each other the opportunity to improve their own score, and could determine each other's weighted deck purely from knowing their own player number and deck weighting.

Introduction

As LLMs become more broad in their application it should be considered whether metric-driven evaluation is an appropriate measure of performance, or if more broad, "thick", evaluation should be considered. Current benchmarking does not consider, for example, whether an LLM approached an answer using sound logic, just that the answer is correct. Similarly, tasks that utilise multiple agents in a multi-agent system (MAS), do not test, for example, if answers are reached due to a dominating singular agent, or if all agents are able to demonstrate their strengths in a problem. These ideas are discussed in (Kommers et al. 2026).

As a first step to start employing a more "thick" method of testing MAS performance, we employed the multiplayer card game *Hanabi* as an experimental benchmark. As a co-operative multi-player hint-based game, we were interested to see how altering the amount of information a player can provide in a given hint can affect the overall performance in the game. Additionally, we modify the game to allow each player to receive it's own score by weighting of deck differently for each player, motivating them to focus on some decks more than others. By combining these two modifications we were interested to see if, when motivations are mixed between a desire for a global success and maximising personal performance, there would be an ideal level of communications between agents that produces the best average score.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Hanabi

Hanabi is a multi-player cooperative card game in which players work together to put on a beautiful fireworks show. The firework show is constructed by placing coloured fireworks cards in order of increasing rank into five separate colour-coordinated decks, with five being the highest value rank in each colour. The total of ranks reached in each colour deck yields the total score for a given game, allowing for a maximum score of 25. Cards can only be played in increasing order, for example a blue rank 3 card can be placed on a blue rank 2 card, but a blue rank 5 card cannot be played on a blue rank 2. If a card is played incorrectly and cannot be placed on any of the decks, the players lose one of three life tokens. If all three tokens are lost then the game is lost. Players each have a hand of a set number of cards (5 for 2-3 players, 4 for 4-5 players), and can see every player's deck but their own. Each turn a player has the choice of playing a card, discarding a card in exchange for an information token, or spending an information token to give a hint to another player about their deck. The hints are constrained in the amount of information they can provide, allowing a player to point out all cards of a given colour in someone's hand, or all cards of a given rank (e.g. "cards 1 and 2 are both green" or "card 2,3, and 4 are all rank 5 cards"). Two possible strategies arise from this mechanic:

1. players play cards that they have complete information on and know confidently can be played
2. players can infer playable cards from which cards players give hints on (e.g. if a player gives a hint that you have one red card, could they be trying to communicate that this red is definitely playable and you should play it next?)

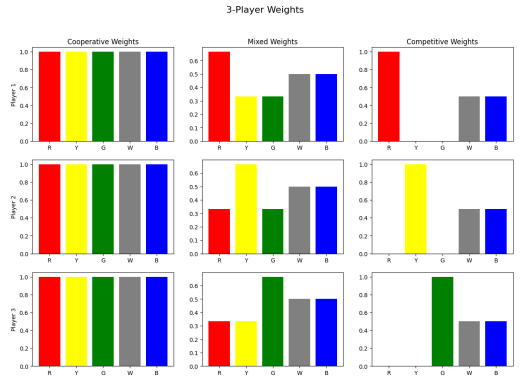


Figure 1: The weights used for the three context levels tested in the three-player games discussed in Section

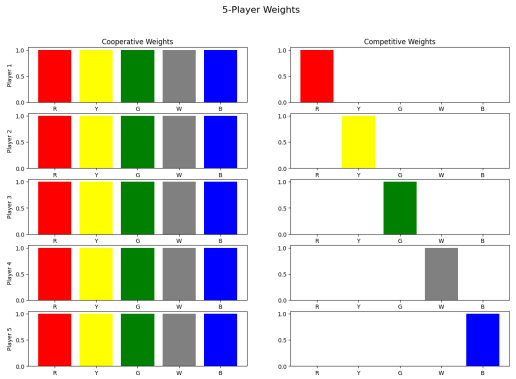


Figure 2: The weights used for the three context levels tested in the three-player games discussed in Section

Level	Included
L0	Hint only
L1	Hint and reasoning
L2	Hint, reasoning, and context

Table 1: The included information for each context level. All other information is trimmed off of the agent output.

The inferred meaning from mechanic 2 is what makes this an interesting game to investigate when considering LLMs - can they infer meaning from the vague hints, and how much do they trust this inferred meaning?

To allow LLMs to interface with this game we use Google DeepMind's code implementation [1]. We make several changes to this code code to adapt it for this experiment. This adapted code is available at `https://github.com/alan-turing-institute/Process_based_MAS_eval.git`.

Firstly, we add a memory mechanic that is fed back to an agent during prompting that contains max_memory number of previous moves. This is a proxy for a human player's ability to recall previous moves made in the game. Since a human would be able to recall that n turns ago a hint was already given to player x about cards α, β, this ability was also given to the agents to improve game performance. We note that this also allows players to see explicitly recent hints that were given, since in the original code the only information available to an agent is the current hands of all other players and a knowledge state on their own cards, with no way to know, for example, how many times or how recently they've recieved a hint on a certain card, or if another player has already received a hint for one of their cards. We create the memory by scraping for an agent's JSON-formatted move choice after they has responded to the prompt for their turn. This is then appended to an array of all moves across the current game. The next player will then receive, as part of their prompt, an array memory$[-max_memory :]$ of most recent moves.

Next, we added the ability to weight the decks such that personal motivations could be injected for each player. These weights for the two setups discussed in this work are shown in Figures 1 and 2.

Finally, we added a context level control that determined how much information a player could provide during a hint. We use the same prompt for all three context levels. Agents are asked to return their move as a dictionary. If they have chosen to hint at a rank or colour, they are asked to additionally output a 'reasoning' and a 'context'. These are then trimmed to include only the information suitable for that context level, as demonstrated in Table 1. This output is then appended to the game memory. For 'reasoning', we prompt the agent to give it's intention for the hint e.g. hinting a card is rank 5 so the player knows not to discard. For 'context', we prompt the agent to explain why they chose this card in specific e.g. this is a high-weighted card in it's personal deck.

[1] https://github.com/google-deepmind/hanabi-learning-environment

Data Extraction

We collect results through several outputs recorded throughout gameplay. These are as follows:

- **api_errors**: any failed attempts to connect to the agent API, recording date and time, number of attempts, rate limit, and error message
- **full_logs**: the complete output from each agent followed by a JSON-formatted summary of their move each turn, and printouts of the current game "board"
- **gameplay**: a JSON-formatted summary of an agent's move each turn, and printouts of the current game "board"
- **logprobs**: if logprobs are enabled, records the logprobs of agent responses each turn
- **stats**: a turn-by-turn record of various game statistics (turn number, current player, action made, life tokens, info. tokens, current deck values, current total score)

The *Hanabi* code developed by Google DeepMind already tracks important game statistics, making extracting things like game score and individual deck scores trivial. However, we were also interested in specific players' playing and hinting of cards, which had to be extracted from the

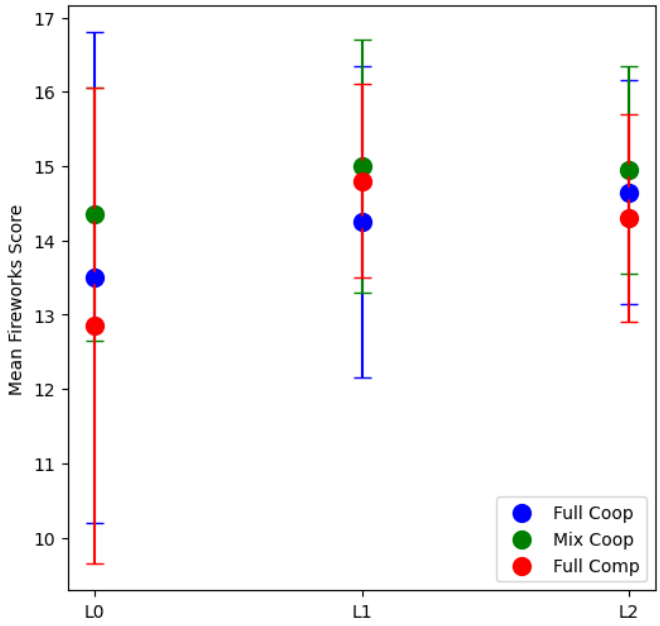


Figure 3: The mean scores across the 20 samples of each game type.

gameplay logs. We enforced during turn prompting that the chosen move from a player should be output in a specific JSON format (see Section). We then extracted card-playing and hint-giving statistics by scraping through the logs for this JSON format[2].

Results

Kimi-K2.5

We first run the experiment using *Kimi-K2.5* as all 3 agents in a 3-player set of games. We ran each of the 9 possible game types 20 times to compare the results of all possible combinations of context levels and weights on the game performance.

We summarise the performance in Figure 3. We can see that while a mixed-cooperative playstyle produces the best performance at all three context levels, all game types produce largely similar game performance. We theorised that this is due to picking a model so large that it hits a performance ceiling where no change made to the game type is significant compared to the model's ability to play the game. To test this we carry out the tests ran in Section with a smaller model and a larger number of players to see if this produces a greater range in performance across game types.

We show the mean reward across turns for each playstyle in Figure 4. Here we take the mean reward across all 20 games for a given playstyle and context level. We note that most plots show a peak in reward followed by a decreasing slope, which we found was explained by games that took a larger number of turns to complete resulting in a lower final reward on average.

Phi-4-reasoning

After finding no significant difference in the performance between game types in Section , we attempted to increase the difficulty to produce a greater range in results. We did this by increasing the number of players to 5 to add complexity and using a smaller model to reduce the reasoning ability. We compare the mean scores achieved by each game type in Figure 5. We can see that competitive games perform better on average regardless of context level, and higher context games typically perform worse. From looking at Figures 7a and 7b we can see there is not a huge difference in the increase in rewards over turns.

To investigate further, we present the following series of plots that investigate how the playing style is affected by the changing game type.

We show in Figure 8 how the agents play card colours depending on the weights and context levels. We can see that the inclusion of weighting (as in, in games that use competitive playstyles) does not impact which cards the players choose to play. This seems to indicate that the weighting does not motivate a difference in behaviour, as one would expect this weighting to cause agents to focus on their own high-weighted deck. However, looking at the hints from the players tells a different story.

In Figure 9 we show the distribution in hints given by each player for each deck colour. We note that this only takes into consideration when a colour hint is given and does not factor in the colours of cards where a rank hint was given. Here we can see that while in the cooperative games (Figures 9a and 9c) there is no significant preference for hinting a specific colour of deck, in the competitive games (Figures 9b and 9d) it is clear that the agents vastly prefer to give hints on their highly-weighted deck compared to the the rest of the decks .

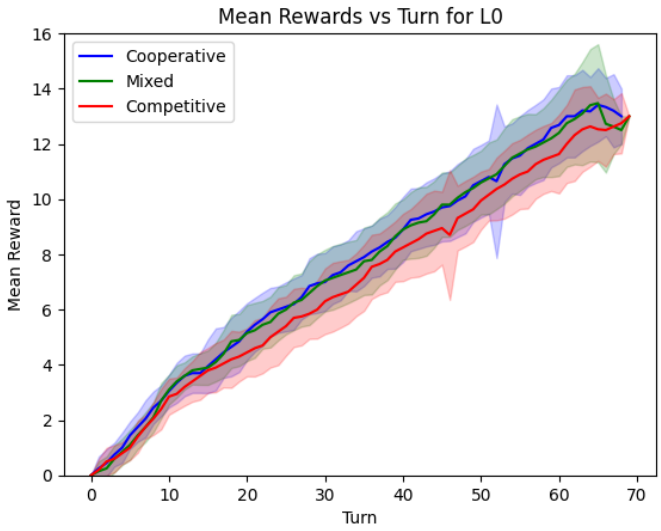
In Figure 10 we show the hints received by each player for each colour. There appears to be a decrease in the competitive games (Figures 10b and 10d) in the hints players received about their higher-weighted deck. This pattern is slightly stronger in Figure 10b at L0 than in 10d at L1.

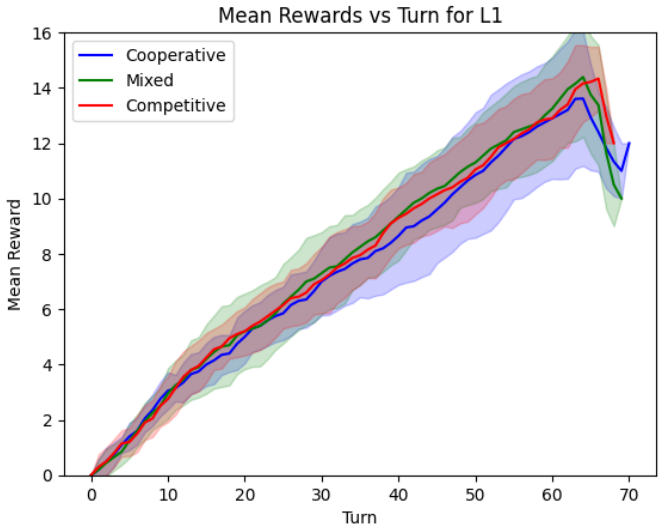
We hypothesised that agents were avoiding giving hints to players about that player's higher-weighted deck. To test for this, we fit a negative binomial (NB) regression with a log link:

$$log\left(\mathbb{E}[n_{p,c}]\right) = \log(N_p) + \beta_0 + \beta_1 \cdot w_{p,c} + \sum_{k=2}^{5} \gamma_k \mathbb{1}[p = k] \tag{1}$$

where N_p is player p's total received hints, β are the coefficients to be fit, $w_{p,c}$ is the weight for player p and colour c, and γ_l is a player-dependent correction for any potential differences in hint rates for specific players across games. We note that this test was not suitable for cooperative games since the weights do not vary and thus cannot be used as a predictor in that case, and so only competitive games are included. We chose to use an NB regression due to an over-dispersion in an initial Poisson regression fit (Pearson

[2]The code for this is available in the Git repository under *gameplay_extraction_code.py*





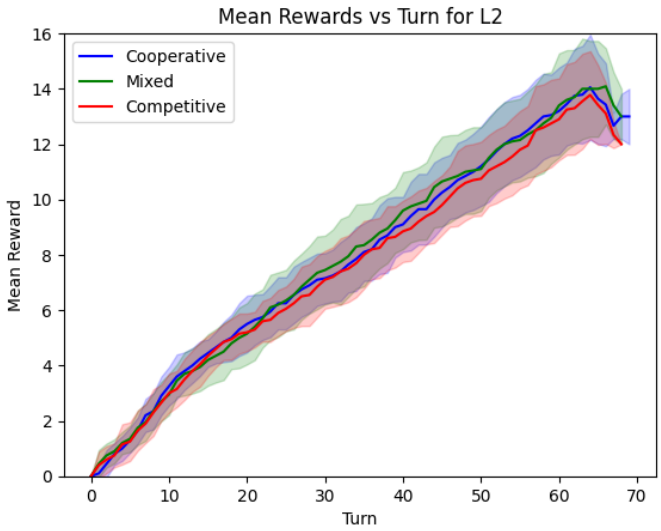


Figure 4: Mean reward taken at each turn across all 20 games of a given context level for all three playstyles, for 3-player Kimi-K2.5 games.

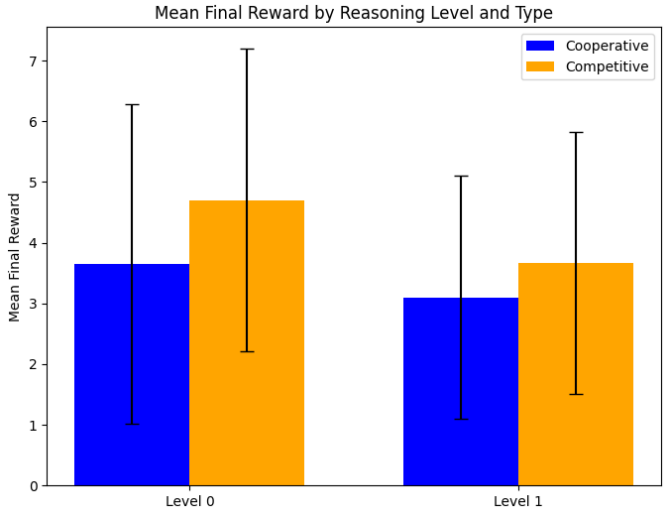


Figure 5: The mean reward for each game time for a 5 player game using the phi-4 model.

Game	α	β	σ	z	p-value
Comp L0	0.0268	-0.8956	0.152	-5.881	<0.0001
Comp L1	0.0107	-0.6450	0.128	-5.051	<0.0001

Table 2: Results from the negative binomial regression

$\chi^2/df = 2.39$). We determined the dispersion parameter for our NB regression using a maximum likelihood estimate.

We determined the one-sided p-values to test if there is a negative effect from the weighting. We summarise the results in Table 2.

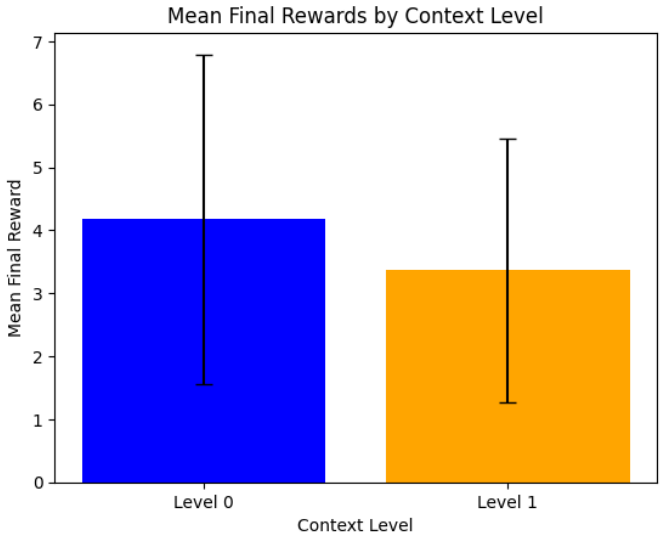
As both p-values are found to be far below the standard significance level of $p < 0.05$, we conclude that a player's weight for a deck $w_{p,c}$ has a strong negative effect on the number of hints that player receives for that deck.

Conclusions

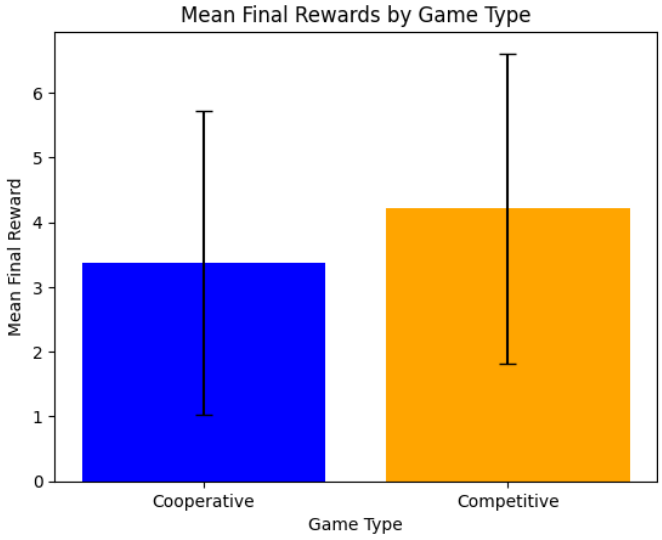
We summarise here several interesting results from this work.

We found that in *Kimi-K2.5* the mixed-weighting playstyles resulted in slightly higher scores than either the cooperative or competitive playstyles. This is an interesting contrast to *phi-4-reasoning* which found competitive playstyles resulted in an average higher score, which was on average the worst-performing playstyle for *Kimi-K2.5*. This may indicate certain playstyles are better suited to different models, but we note that these results are not statistically significant and these conclusions could only be confirmed by running a greater sample size of games. Additionally, we did not run a mixed-weighting playstyle for *phi-4-reasoning*, and future tests may reveal that mixed-weighting produces the best results regardless of model.

We found in our investigation into *phi-4-reasoning* gameplay that despite playing competitively, players did not preferentially play cards corresponding to their weighted deck. We expect this is explained by our findings that players pref-

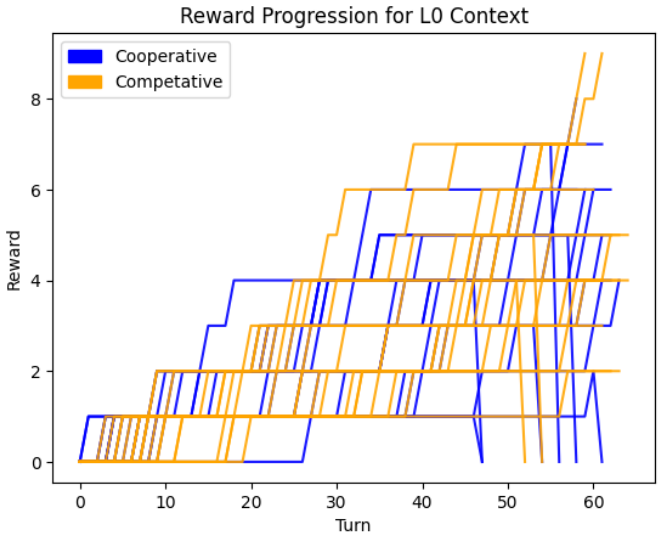


(a) The mean final reward for context levels 0 and 1.

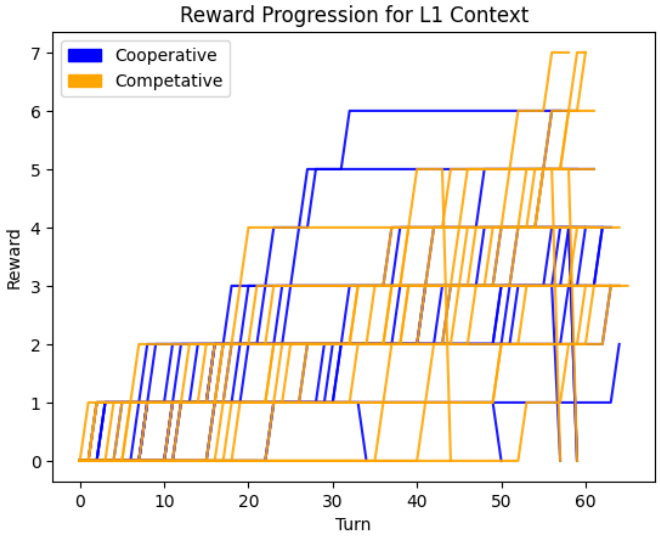


(b) The mean final reward across for cooperative and competitive play styles.

Figure 6: The mean final reward averaged across play style for each context level (a) and averaged across context levels for cooperative and competitive play styles (b)

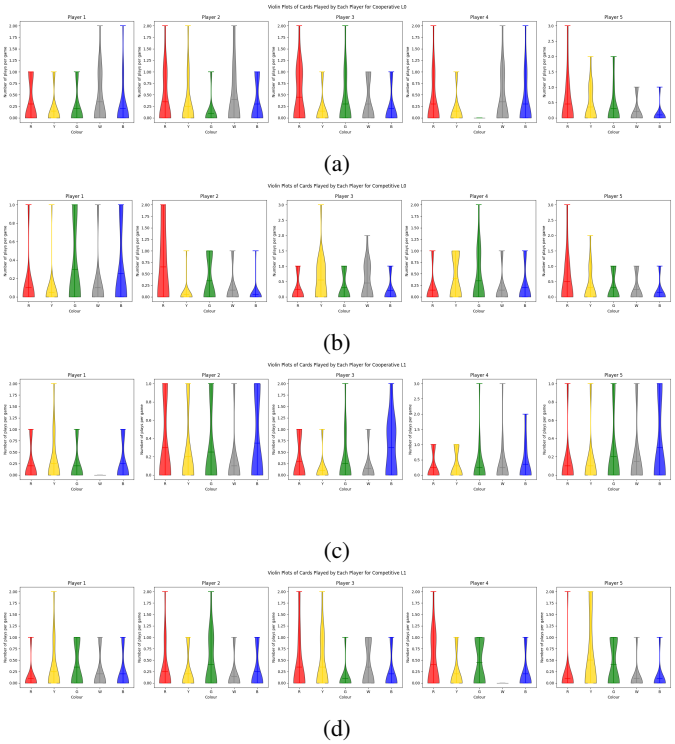


(a)



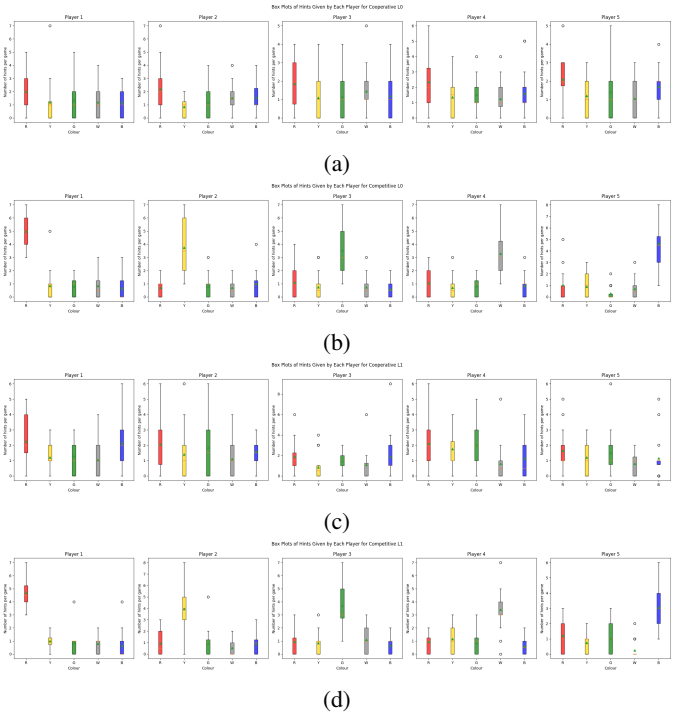
(b)

Figure 7: The reward against turn for all games with a context level of 0 (a) and 1 (b)



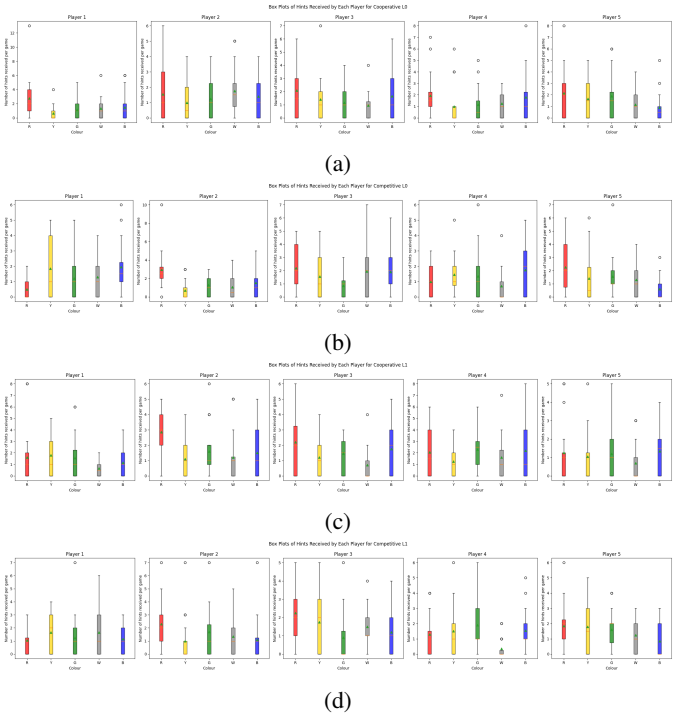
(a) (b) (c) (d)

Figure 8: Violin plots demonstrating the distribution of cards played to each deck by each player across all games.



(a) (b) (c) (d)

Figure 9: Box plots demonstrating the distribution of hints given for each deck by each player across all games.



(a) (b) (c) (d)

Figure 10: Box plots demonstrating the distribution of hints given for each deck by each player across all games.

erentially give hints about their own decks to other players when playing competitively, and also *avoid* giving hints to players about a deck that the receiving player weights highly. We find this last discovery a particularly interesting result, as this implies agents are able to infer that other players will preferentially play their own weighted deck if given the opportunity, and so agents try to avoid providing this opportunity. This also implies that players were able to infer each other's weighted deck even under circumstances where they could not have been told (i.e. in L0 games where very little information is passed between players), and thus the only information they had to make this inference was their *own* deck weighting and player number, since the preferred deck number corresponds to the player number.

Limitations and Future Work

We note that each agent/player uses the same model with no change in prompting or persona, and thus this work does not examine the performance of a true multi-agent system. Future works can investigate the effect of different prompting (e.g. encouraging different playstyles in different agent) on the outcome of the game, as well as using different models for each player.

We recommend future work uses a midsize (10^{10}s of parameters) non-reasoning model for future work. Since *Kimi-K2.5* is a very large, reasoning-enabled model and *phi-4-reasoning* is a small, reasoning-enabled model, we suspect this recommended model type will find a comfortable middle ground between the uniformly high-performing large

models and prohibitively poor-performing small models.

Many results in this work were not conclusively due to the wide error margins on metrics like game score. We would encourage future work to increase the number of game samples for each game type from 20 to 100 to allow for greater precision in results.

We encourage future investigations to test for emergent tactful behaviour such as avoiding hinting to players about their own highly-weighted deck with other large-language models.

References

[Kommers et al. 2026] Kommers, C.; Ahnert, R.; Antoniak, M.; Benetos, E.; Benford, S.; Bunz, M.; Caramiaux, B.; Concannon, S.; Disley, M.; Dobson, J.; Du, Y.; Duéñez-Guzmán, E.; Francksen, K.; Gius, E.; Gray, J. W. Y.; Heuser, R.; Immel, S.; So, R. J.; Leigh, S.; Livingston, D.; Long, H.; Martin, M.; Meyer, G.; Mihai, D.; Noel-Hirst, A.; Ostherr, K.; Parker, D.; Qin, Y.; Ratcliff, J.; Robinson, E.; Rodriguez, K.; Sobey, A.; Underwood, T.; Vashistha, A.; Wilkens, M.; Wu, Y.; Zheng, Y.; and Hemment, D. 2026. Computational hermeneutics: evaluating generative ai as a cultural technology. *Frontiers in Artificial Intelligence* 9.

Appendices

Appendix A: Prompts

Agent Intialisation Prompt

You are a player in a n_player-player cooperative game of Hanabi. Your team scores points by playing cards to build five fireworks (R, Y, G, W, B), each requiring cards played in order from rank 0 to rank 4.

IMPORTANT RULES: - You CANNOT see your own cards, but you CAN see your teammates' cards. - A card is playable if its rank equals the current fireworks value for its color (e.g. if fireworks R=2, you need rank 2 to play next on red). - Information tokens are spent to give hints and recovered by discarding. Discarding when tokens are full would be wasteful. - Life tokens are lost when an unplayable card is played (e.g. if you play a blue rank 3 card when fireworks B=4). You only have 3 life tokens. - Score is determined by adding together the current fireworks values (e.g. if fireworks are R=2, Y=1, G=0, W=3, B=1, score is 2+1+0+3+1=7). The maximum score is 25.

Agent Turn Prompt

"The current state of the game is as follows: Firework deck status: R=fireworks['R'], Y=fireworks['Y'], G=fireworks['G'], W=fireworks['W'], B=fireworks['B']. This is a summary of the current fireworks piles, where each value represents the NEXT RANK NEEDED FOR THAT COLOR. For example, if fireworks B=2, that means the next card needed to play on the blue pile is rank 2. Life tokens: obs['life_tokens'] — Info tokens: obs['information_tokens'] Your hand (what you know): my_knowledge Teammates' hands: teammate_hands Discard pile: obs['discard_pile'] Legal moves: obs['legal_moves']].

Here are the previous max_memory moves made in the game: memory[$-max_memory$:]. Use this history to inform your understanding of the other players' strategies and how they might interpret your actions.

Choose your next action. This must be one of the following: 1. PLAY: You need to specify the index of the card in your hand that you want 2. DISCARD: You need to specify the index of the card in your hand that you want to discard 3. REVEAL_COLOR: You need to specify the target player and the color you want 4. REVEAL_RANK: You need to specify the target player and the rank you want to reveal Only output the action in the following format: For PLAY and DISCARD: 'action_type': 'PLAY', 'card_index': 0 or 'action_type': 'DISCARD', 'card_index': 3 For REVEAL_COLOR: 'action_type': 'REVEAL_COLOR', 'color': 'R', 'target_offset': 1, 'reasoning': <reasoning>, 'context': <context> where target_offset refers to the player shown as "Player at target_offset=1" in your observation. For REVEAL_RANK: 'action_type': 'REVEAL_RANK', 'rank': 3, 'target_offset': 2, 'reasoning': <reasoning>, 'context': <context> where target_offset refers to the player shown as "Player at target_offset=2" in your observation. When asked for <reasoning>: provide one sentence explaining your intention behind giving this hint. For example, if you give a REVEAL_RANK hint to player at offset=2, it might be because there is only one deck that requires a rank 2 card, and you are trying to communicate to the other player this card is playable. Therefore your reasoning might be "I want to signal to player at offset=2 that they have a playable card in their hand, which is the only card that can be played on the fireworks at the moment, so they know it is safe to play this card on their next turn." When asked for <context>: provide a sentence on your motivation for picking this deck specifically. For example, if each player has a playable card but one of the players has a playable card for a deck that is higher weighted for you, you might want to hint that card to increase the chances of that deck being completed, which would increase your personal score. In this case your context might be "I want to increase the chances of the red fireworks being completed soon, because red has the highest deck weighting for my personal score among currently playable colors.

First, state what you currently believe your hidden cards are, with your confidence for each. Before choosing your action, reason through: 1. Can I see any immediately playable cards in my teammate's hand? 2. Does my card_knowledge tell me any of my cards are definitely playable? 3. Are info tokens low enough that I should discard instead of hint? Then output ONLY the action dict."